

Implementasi Struktur Data Tree pada Program Pengelolaan Organisasi Kelas

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL) Mata
Kuliah Algoritma & Struktur Data (TPL2106)



Disusun oleh:

Nama NIM

Ibra Arifa Istara J0403251029

Muhammad Adiyoga Danendra J0403251063

Ayyash Syauqi Syahadah J0403251100

Program Studi Teknologi Rekayasa Perangkat Lunak

Sekolah Vokasi IPB University

2026

KATA PENGANTAR

Puji syukur kami panjatkan kehadiran Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, kami dapat menyelesaikan laporan tugas besar yang berjudul "**Implementasi Struktur Data Tree pada Program Pengelolaan Organisasi Kelas**" ini dengan baik dan tepat waktu.

Laporan ini disusun untuk memenuhi salah satu tugas kelompok pada mata kuliah Algoritma dan Struktur Data. Dalam program yang kami rancang, kami mengimplementasikan konsep struktur data non-linear (*Tree*) serta algoritma pengurutan (*Insertion Sort*) untuk menyelesaikan permasalahan manajemen data organisasi secara digital dan sistematis.

Melalui penyusunan laporan dan pembuatan program ini, kami mendapatkan banyak wawasan baru mengenai bagaimana konsep-konsep teoritis dalam perkuliahan dapat diimplementasikan ke dalam kode program nyata guna menyelesaikan masalah struktural di dunia nyata.

Kami menyadari bahwa keberhasilan penyusunan laporan ini tidak lepas dari bantuan berbagai pihak. Oleh karena itu, kami ingin menyampaikan terima kasih kepada Bu Dr. Shelve Nidya Neyman, selaku dosen mata kuliah.

Kami menyadari bahwa program dan laporan yang kami buat ini masih jauh dari kata sempurna. Oleh karena itu, kami sangat mengharapkan kritik, saran, dan masukan yang membangun dari pembaca demi perbaikan di masa yang akan datang. Akhir kata, semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi kita semua, khususnya dalam pemahaman materi Algoritma dan Struktur Data.

Bogor, 26 Juni 2026

DAFTAR ISI

KATA PENGANTAR	2
DAFTAR ISI	3
BAB I. PENDAHULUAN	5
1.1 Latar Belakang	5
1.2 Tujuan Proyek	5
1.3 Ruang Lingkup	6
BAB II. ANALISIS DAN PERANCANGAN	7
2.1 Deskripsi Sistem	7
2.2 Struktur Data yang Digunakan	7
2.3 Algoritma yang Digunakan	8
2.4 Diagram Alir Program (Flowchart)	9
2.5 Struktur Program	9
BAB III. IMPLEMENTASI PROGRAM	11
3.1 Tampilan Menu Utama	11
3.2 Fitur Create	11
3.3 Fitur Read	12
3.4 Fitur Update	12
3.5 Fitur Delete	13
3.6 Fitur Searching	13
3.7 Fitur Sorting	14
3.8 Penyimpanan Data (File Handling)	14
BAB IV. PENGUJIAN DAN ANALISIS	17
4.1 Skenario Pengujian	17
4.2 Kelebihan Sistem	17
4.4 Keterbatasan Sistem	18
BAB V. KENDALA DAN SOLUSI	19
BAB VI. PEMBAGIAN TUGAS ANGGOTA	20
BAB VII. KESIMPULAN DAN SARAN	21

BAB I. PENDAHULUAN

1.1 Latar Belakang

Pengelolaan struktur organisasi kelas pada umumnya masih dilakukan secara manual, baik dalam bentuk catatan tertulis maupun dokumen sederhana yang tidak terstruktur. Cara tersebut memiliki beberapa kelemahan, antara lain data yang mudah hilang, sulit diperbarui ketika terjadi perubahan jabatan atau anggota, serta tidak adanya hierarki yang jelas antara satu jabatan dengan jabatan lainnya. Permasalahan tersebut mendorong perlunya suatu sistem yang dapat menyimpan dan mengelola data organisasi kelas secara terstruktur dan permanen.

Proyek ini dibuat untuk menjawab permasalahan tersebut dengan membangun aplikasi berbasis terminal menggunakan bahasa pemrograman Python. Aplikasi ini memanfaatkan struktur data Tree untuk merepresentasikan hubungan hierarkis antarjabatan dalam organisasi kelas, mulai dari ketua kelas sebagai pemimpin tertinggi hingga anggota pada tingkat yang lebih rendah. Selain itu, aplikasi ini juga menerapkan struktur data Queue untuk mencatat riwayat anggota yang telah dihapus dari struktur organisasi. Seluruh data disimpan dalam format JSON sehingga informasi yang telah dimasukkan tetap tersedia meskipun program telah dihentikan dan dijalankan kembali di waktu yang berbeda.

1.2 Tujuan Proyek

Tujuan dari proyek ini adalah sebagai berikut.

1. Membangun aplikasi berbasis terminal menggunakan bahasa pemrograman Python untuk mengelola struktur organisasi kelas.
2. Mengimplementasikan struktur data Tree dan Queue, masing-masing untuk merepresentasikan hierarki jabatan dan mencatat riwayat penghapusan anggota.
3. Menerapkan penyimpanan data berbasis file dalam format JSON serta menyediakan fitur penambahan, penghapusan, pengubahan, dan pencarian data anggota organisasi.

1.3 Ruang Lingkup

- Platform CLI (Console)
- Bahasa Python
- Penyimpanan file JSON
- Fitur CRUD
- DFS Searching
- Insertion Sorting

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Sistem yang dibuat adalah aplikasi pengelola struktur organisasi kelas. Sistem memungkinkan pengguna membuat struktur organisasi kelas. Selain itu, sistem dapat menambah, mengubah, menghapus, dan mencari data anggota pada struktur tersebut. Data disimpan pada file json sehingga tetap tersedia ketika program dijalankan kembali.

2.2 Struktur Data yang Digunakan

Struktur Data	Fungsi
Tree	Menyimpan data struktur organisasi kelas
Queue	Mengelola riwayat anggota organisasi kelas

Alasan Pemilihan Struktur Data

Tree adalah struktur data hierarkis yang terdiri dari node-node dengan hubungan parent-child, dengan satu node tertinggi (root) yang tidak memiliki parent [1]. Struktur data tree dipilih karena sangat sesuai untuk merepresentasikan hubungan hierarkis yang ada pada organisasi kelas. Struktur ini menempatkan ketua kelas sebagai root di puncak hierarki, yang kemudian bercabang ke bawah menjadi child nodes seperti wakil ketua, sekretaris, dan bendahara. Struktur kepengurusan ini terus bercabang hingga mencapai seksi bidang tertentu dan anggotanya sehingga hubungan antarjabatan dapat dipetakan secara akurat.

Queue adalah struktur data yang menerapkan prinsip First In First Out (FIFO), di mana elemen yang pertama masuk akan menjadi elemen pertama yang dikeluarkan [2]. Struktur data queue dipilih karena sangat sesuai untuk merepresentasikan daftar riwayat anggota. Ketika ada anggota baru yang dihapus, maka anggota tersebut akan muncul paling awal dan anggota yang masuk ke daftar riwayat terlebih dahulu akan dihapus dari daftar, hal ini sesuai dengan prinsip First In First Out pada queue.

2.3 Algoritma yang Digunakan

Searching

Depth-First Search

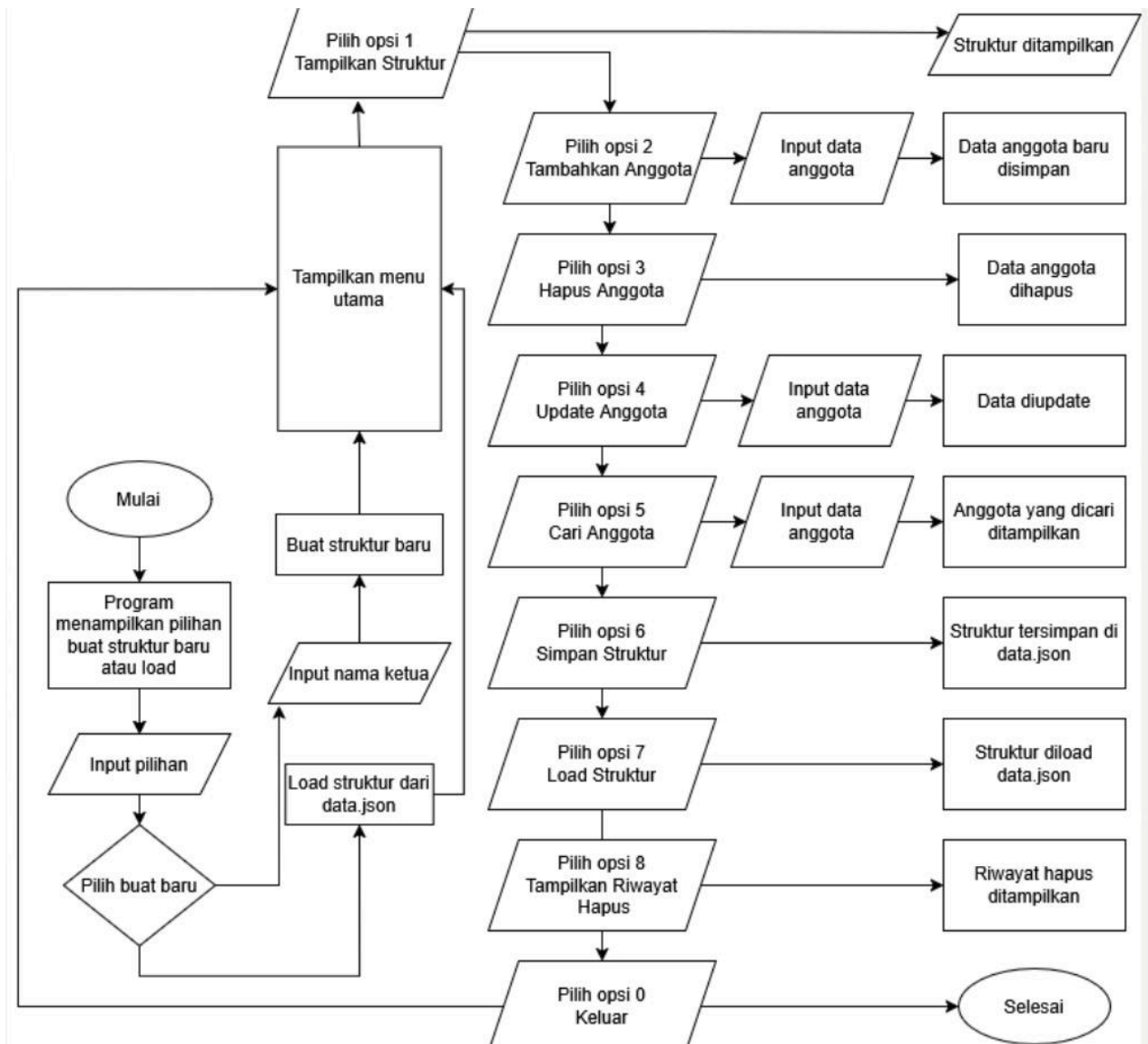
DFS adalah algoritma penelusuran rekursif yang menyusuri satu cabang tree/graph secara penuh sebelum berpindah ke cabang lain [3]. Algoritma ini dipilih karena data organisasi disimpan dalam bentuk struktur Tree, sehingga pencarian terhadap anggota tertentu memerlukan penelusuran dari node akar menuju ke node-node turunannya hingga ditemukan nama yang sesuai dengan target pencarian. Pendekatan rekursif pada DFS memungkinkan setiap node, beserta seluruh bawahannya, ditelusuri secara menyeluruh tanpa perlu mengubah struktur data ke dalam bentuk lain seperti array atau list, sehingga hierarki jabatan pada organisasi tetap terjaga selama proses pencarian berlangsung.

Sorting

Insertion Sort

Insertion sort adalah algoritma pengurutan yang menyisipkan satu elemen ke posisi yang tepat dalam bagian data yang sudah terurut, diulang untuk setiap elemen [4]. Insertion sort dipilih untuk mengurutkan bawahan suatu siswa secara alfabetis ketika menambahkan anggota. Insertion sort digunakan karena cocok untuk data yang pendek. Sehingga, sesuai dengan program ini, karena setiap siswa mempunyai bawahan yang sedikit (sekitar 1- 10 orang).

2.4 Diagram Alir Program (Flowchart)



2.5 Struktur Program

main.py

main.py berfungsi sebagai kode program utama yang berisi class struktur data dan fungsi-fungsi yang dibutuhkan untuk menjalankan program.

data.json

data.json berfungsi sebagai tempat menyimpan struktur kelas agar tidak ter-reset ketika program main.py berhenti.

BAB III. IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama

```
=====
===== Struktur Organisasi Kelas =====
=====
1. Tampilkan Struktur Organisasi
2. Tambah Anggota
3. Hapus Anggota
4. Update Anggota
5. Cari Anggota
6. Simpan Struktur
7. Load Struktur
8. Tampilkan Riwayat Hapus
0. Keluar
Masukkan pilihan:
```

3.2 Fitur Create

Fitur Create pada program ini diimplementasikan melalui opsi Tambah Anggota. Saat dijalankan, opsi ini akan meminta tiga input dari pengguna: nama atasan, nama anggota baru, dan jabatannya. Di dalam sistem, data anggota baru tersebut akan diproses dan disisipkan sebagai child dari node atasan yang telah ditentukan dalam struktur Tree.

```
0. Keluar
Masukkan pilihan: 2
Masukkan nama atasan: Andi
Masukkan nama anggota baru: Budi
Masukkan jabatan anggota baru: Wakil
Berhasil menambahkan Budi sebagai Wakil.
```

3.3 Fitur Read

Fitur Read pada aplikasi ini diimplementasikan melalui menu Tampilkan Struktur Organisasi. Saat dipilih, program akan menyusuri seluruh tree secara rekursif mulai dari ketua kelas sebagai akar, kemudian menampilkan setiap anggota beserta jabatannya secara berjenjang sesuai dengan tingkat hierarkinya. Setiap tingkat kedalaman ditandai dengan indentasi, sehingga hubungan atasan dan bawahan dapat terlihat secara visual.

```
--- Struktur Organisasi Kelas ---
Yanto (Ketua Kelas)
  |_ Wawan (Wakil Ketua)
    |_ Ucup (Sekretaris)
      |_ Ucok (Wakil Sekretaris)
    |_ Saipul (Bendahara)
  |_ Adi (Siswa)
  |_ Arifa (Siswa)
  |_ Syauqi (Siswa)
  |_ Rizky (Siswa)
  |_ Jamal (Siswa)
```

3.4 Fitur Update

Fitur Update pada program ini terdapat pada opsi Update Anggota. Fitur ini bekerja dengan cara mencari nama anggota yang ditargetkan di dalam struktur Tree secara rekursif, kemudian memberikan fleksibilitas kepada pengguna untuk memperbarui nama baru, jabatan baru, atau keduanya sekaligus. Program akan mempertahankan data lama yang sudah ada dan hanya memperbarui bagian yang diisi tanpa merusak relasi hierarki node tersebut di dalam organisasi.

```
Masukkan pilihan: 4
Masukkan nama anggota yang ingin diubah: Adi

(Kosongkan dan tekan Enter pada bagian yang tidak ingin diubah)
Masukkan nama baru: Adi Firmansyah
Masukkan jabatan baru:
Berhasil memperbarui data 'Adi'.
```

3.5 Fitur Delete

Fitur Delete pada program ini diimplementasikan melalui opsi Hapus Anggota. Fitur ini menerapkan sistem cascading delete, di mana penghapusan suatu node akan menghapus seluruh sub-tree (bawahan) di bawahnya. Program ini dilengkapi dengan proteksi yang mencegah penghapusan Ketua Kelas selaku root. Selain itu, setiap data yang dihapus akan dialirkan ke dalam struktur data Queue (deque) pada kelas Riwayat.

```
Masukkan pilihan: 3
Masukkan nama anggota yang ingin dihapus: Ukok
Berhasil menghapus 'Ukok' beserta seluruh bawahannya dari struktur.
```

3.6 Fitur Searching

Fitur Searching pada program ini terdapat pada opsi Cari Anggota. Proses pencarian ini menerapkan algoritma Tree Traversal secara rekursif, di mana sistem akan menelusuri setiap simpul (node) mulai dari Ketua Kelas (root) hingga ke tingkat bawahan. Selain itu, fitur pencarian ini bersifat case-insensitive (menggunakan fungsi `.lower()`) sehingga proses pencocokan nama tetap akurat dan fleksibel tanpa terpengaruh oleh penggunaan huruf besar atau kecil yang diinput oleh pengguna.

```
Masukkan pilihan: 5
Masukkan nama anggota yang ingin dicari: Yanto

===Data Anggota===
Nama      : Yanto
Jabatan   : Ketua Kelas
Bawahan:
- Wawan (Wakil Ketua)
```

3.7 Fitur Sorting

Fitur Sorting pada program ini menggunakan algoritma *Insertion Sort*, yang dijalankan secara otomatis setiap kali ada anggota baru ditambahkan. Pengurutan dilakukan berdasarkan nama anggota secara alfabetis terhadap daftar bawahan dari atasan yang sama.

```
Masukkan pilihan: 2
Masukkan nama atasan: Yanto
Masukkan nama anggota baru: Wawan
Masukkan jabatan anggota baru: Wakil
Berhasil menambahkan Wawan sebagai Wakil.
```

```
Masukkan pilihan: 2
Masukkan nama atasan: Yanto
Masukkan nama anggota baru: Rahmat
Masukkan jabatan anggota baru: Bendahara
Berhasil menambahkan Rahmat sebagai Bendahara.
```

```
--- Struktur Organisasi Kelas ---
Yanto (Ketua Kelas)
  |_ Rahmat (Bendahara)
  |_ Wawan (Wakil)
```

3.8 Penyimpanan Data (File Handling)

Program ini memanfaatkan file JSON untuk keperluan *file handling*. Proses pembacaan dilakukan dengan memuat file JSON, lalu mengonversi data mentah tersebut ke dalam bentuk objek *Tree*. Sebaliknya, saat proses penyimpanan, seluruh hierarki struktur data *Tree* yang aktif di memori akan dikonversi kembali ke dalam format teks JSON agar data tetap tersimpan secara permanen meskipun program ditutup.

```
1  {
2    "nama": "Yanto",
3    "jabatan": "Ketua Kelas",
4    "bawahan": [
5      {
6        "nama": "Wawan",
7        "jabatan": "Wakil Ketua",
8        "bawahan": [
9          {
10             "nama": "Ucup",
11             "jabatan": "Sekretaris",
12             "bawahan": [
13               {
14                 "nama": "Ucok",
15                 "jabatan": "Wakil Sekretaris",
16                 "bawahan": []
17               }
18             ]
19           },
20           {
21             "nama": "Saipul",
22             "jabatan": "Bendahara",
23             "bawahan": []
24           },
25           {
26             "nama": "Adi",
27             "jabatan": "Siswa",
28             "bawahan": []
29           }
30         ]
31       }
32     ]
33   }
```

3.9 Fitur Riwayat

Fitur Riwayat pada program ini menggunakan kelas Riwayat yang memanfaatkan struktur data Queue berbasis deque dengan kapasitas maksimal lima elemen. Setiap kali fungsi delete dijalankan, objek anggota yang dihapus beserta seluruh bawahannya akan dikonversi menjadi dictionary lalu dimasukkan ke baris antrean terdepan. Program akan membaca antrean tersebut untuk menampilkan lima data anggota terakhir yang telah dihapus.

```
===5 Anggota Terakhir yang Dihapus===  
1.  
Arifa (Siswa)  
2.  
Adi (Siswa)  
3.  
Ucok (Wakil Sekretaris)
```

3.10 Validasi Input dan Error Handling

Kesalahan Input	Penanganan
Huruf pada input angka	Program meminta input ulang
Data tidak ditemukan	Pesan error ditampilkan
File belum tersedia	Program membuat file baru

BAB IV. PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

No	Skenario	Hasil
1	Tambah data	Berhasil
2	Edit data	Berhasil
3	Hapus data	Berhasil
4	Sorting data	Berhasil
5	Searching data	Berhasil

4.2 Kelebihan Sistem

1. Data tersimpan permanen.

Data struktur kelas tetap tersimpan di file json meskipun program utamanya berhenti.

2. Program tidak mudah crash.

Program dibuat dengan mempertimbangkan segala kemungkinan input yang dilakukan oleh user, sehingga dapat ditangani dengan semua error handler yang tersedia di program tersebut.

3. Menu mudah digunakan.

Menu utama memakai bahasa yang mudah dipahami sehingga mudah digunakan.

4.4 Keterbatasan Sistem

1. Masih berbasis CLI.

Antarmuka program masih berupa command line dan program dijalankan di terminal. Program tidak cocok digunakan oleh pengguna yang terbiasa memakai antarmuka berbasis grafis.

2. Belum mendukung database.

Program masih mengandalkan json untuk menyimpan data

BAB V. KENDALA DAN SOLUSI

Kendala	Solusi
Data hilang saat program ditutup	Menyimpan data pada file json
Program error saat input salah	Membuat kode yang berfungsi sebagai error handler
Sorting berjalan lambat	Mengganti algoritma yang lebih efisien

Terdapat beberapa rintangan yang kami hadapi saat membuat program ini. Salah satunya ada pada fungsi `hapus_anggota()`. Awalnya kami berniat untuk membuat sistem hapus anggota dimana bawahan dari anggota tersebut akan menggantikan anggota yang dihapus. Namun, kami berpikir bahwa pada struktur kelas di dunia nyata, anggota yang sudah memiliki jabatan tertentu tidak bisa asal diubah jabatannya hanya untuk menggantikan atasannya. Sebagai solusi, kami memutuskan untuk menghapus seluruh bawahan dari anggota yang dihapus.

BAB VI. PEMBAGIAN TUGAS ANGGOTA

Nama	Tugas
Ibra Arifa Istara	Membuat struktur data utama (tree)
Muhammad Adiyoga Danendra	Membuat fungsi main() dan antarmuka
Ayyash Syauqi Syahadah	Membuat fungsi yang menghubungkan file utama dan json

BAB VII. KESIMPULAN DAN SARAN

Kesimpulan

Berdasarkan apa yang sudah kami sampaikan, proyek ini telah berhasil diimplementasikan dengan membangun aplikasi berbasis CLI menggunakan bahasa pemrograman Python untuk pengelolaan struktur organisasi kelas. Melalui penerapan struktur data tree, sistem mampu merepresentasikan hierarki jabatan secara sistematis dan terstruktur, yang didukung oleh penggunaan struktur data queue untuk manajemen riwayat penghapusan anggota secara efisien. Penggunaan format penyimpanan json berhasil menjamin permanennya data, sehingga seluruh informasi organisasi tetap terjaga dan tidak hilang saat program dihentikan.

Fitur CRUD telah berfungsi optimal sesuai dengan skenario pengujian yang ditetapkan. Implementasi algoritma Depth-First Search (DFS) untuk proses pencarian dan insertion sort untuk pengurutan data telah berjalan efektif, didukung oleh mekanisme error handling yang menjaga stabilitas program terhadap kesalahan input pengguna..

Saran Pengembangan

Adapun saran yang dapat kami sampaikan untuk pengembangan kedepannya adalah untuk menambahkan fitur-fitur sebagai berikut:

1. Pengembangan antarmuka GUI dengan library seperti Tkinter atau PyQt.
 2. Penggunaan DBMS untuk menggantikan json.
 3. Penambahan sistem login agar struktur kelas hanya bisa diakses oleh individu yang berkepentingan dan mencegah penyalahgunaan data.
-

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Introduction to tree data structure," *GeeksforGeeks*, Oct. 7, 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/introduction-to-tree-data-structure/>. [Accessed: Jun. 25, 2026].
- [2] GeeksforGeeks, "Queue data structure," *GeeksforGeeks*, Jan. 20, 2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/queue-data-structure/>. [Accessed: Jun. 25, 2026].
- [3] GeeksforGeeks, "Depth first search or DFS for a graph," *GeeksforGeeks*, Mar. 28, 2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>. [Accessed: Jun. 25, 2026].
- [4] GeeksforGeeks, "Insertion sort algorithm," *GeeksforGeeks*, Feb. 24, 2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/>. [Accessed: Jun. 25, 2026].
-

LAMPIRAN

Lampiran A. Link GitHub

https://github.com/spotlessoutset/ASD_A_22.git

Lampiran B. Screenshot Program

```
=====
===== Struktur Organisasi Kelas =====
=====
1. Tampilkan Struktur Organisasi
2. Tambah Anggota
3. Hapus Anggota
4. Update Anggota
5. Cari Anggota
6. Simpan Struktur
7. Load Struktur
8. Tampilkan Riwayat Hapus
0. Keluar
Masukkan pilihan: 1

--- Struktur Organisasi Kelas ---
Yanto (Ketua Kelas)
|_ Wawan (Wakil Ketua)
|_ Ucup (Sekretaris)
|_ Ucok (Wakil Sekretaris)
|_ Saipul (Bendahara)
|_ Adi (Siswa)
|_ Arifa (Siswa)
|_ Syauqi (Siswa)
|_ Rizky (Siswa)
|_ Jamal (Siswa)
```

Lampiran C. Struktur Folder Proyek

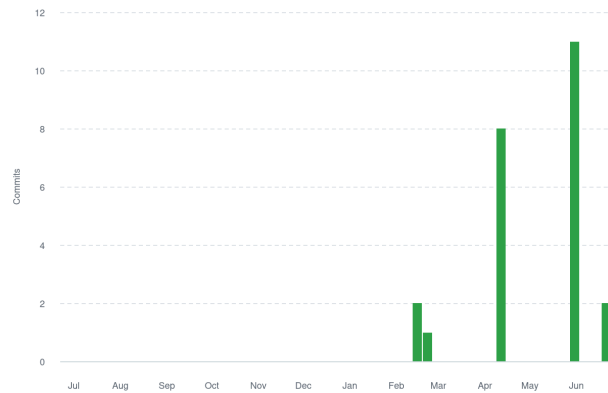
 AdiyogaD Update README.md 2c9293f · 4 hours ago  27 Commits		
 README.md	Update README.md	4 hours ago
 data.json	insertion sort	3 weeks ago
 main.py	insertion sort	3 weeks ago

Lampiran D. Commit History GitHub

https://github.com/spotlessoutset/ASD_A_22/commits/main/

Commits

Number of commits per week



Lampiran E. Source Code Utama

```
# =====  
# STRUKTUR ORGANISASI KELAS  
# KELOMPOK : 22  
# ANGGOTA : 1. Ibra Arifa Istara  
#           2. Ayyash Syauqi Syahadah  
#           3. Muhammad Adiyoga Danendra  
# =====  
  
from collections import deque  
import json  
  
# Mengubah objek Anggota menjadi dictionary  
def anggota_ke_dict(node):  
    return {  
        "nama": node.nama,  
        "jabatan": node.jabatan,  
        "bawahan": [anggota_ke_dict(b) for b in node.bawahan]  
    }  
  
# Mengubah dictionary kembali menjadi objek Anggota
```

```

def dict_ke_anggota(data):
    node = Anggota(data["nama"], data["jabatan"])
    node.bawahan = [dict_ke_anggota(b) for b in data["bawahan"]]
    return node

# 5.
# Menyimpan struktur ke file JSON
def simpan_file(organisasi):
    if organisasi.root is None:
        print("Tidak ada data untuk disimpan.")
        return

    data = anggota_ke_dict(organisasi.root)

    try:
        with open("data.json", "w") as file:
            json.dump(data, file, indent=4)
        print("Struktur berhasil disimpan ke 'data.json'")
    except Exception as e:
        print("Terjadi kesalahan saat menyimpan:", e)

# 6.
# Membuka struktur dari file JSON
def buka_file(organisasi):
    try:
        with open("data.json", "r") as file:
            data = json.load(file)

            organisasi.root = dict_ke_anggota(data)
            print("Struktur berhasil dimuat dari 'data.json'")
            return True
    except FileNotFoundError:
        print("File tidak ditemukan!")
    except Exception as e:
        print("Terjadi kesalahan saat memuat:", e)
        return False

# =====

```



```

# Class Node
# =====

# Merepresentasikan satu anggota dalam struktur organisasi.
class Anggota:
    # Inisialisasi anggota dengan nama, jabatan, dan list bawahan
    # kosong
    def __init__(self, nama, jabatan):
        self.nama = nama
        self.jabatan = jabatan
        self.bawahan = []

# =====
# Class Riwayat
# =====

# Menampilkan 5 anggota terakhir yang dihapus
class Riwayat:
    def __init__(self):
        self.queue = deque(maxlen=5)

    def tambah(self, node):
        self.queue.appendleft(anggota_ke_dict(node))

    def tampilkan(self):
        if not self.queue:
            print("Belum ada anggota yang dihapus.")
            return

        # Rekursif buat nampilin node beserta seluruh bawahannya
        def tampilkan_node(data, tingkat=0):
            indentasi = "    " * tingkat + ("|_ " if tingkat > 0
            else "")
            print(f"{indentasi}{data['nama']}
            ({data['jabatan']})")
            for bawahan in data['bawahan']:
                tampilkan_node(bawahan, tingkat + 1)

```

```

        print("\n==5 Anggota Terakhir yang Dihapus==")
        for i, data in enumerate(self.queue, 1):
            print(f"{i}.")
            tampilkan_node(data)

# =====
# Class Struktur Data
# =====

# Mengelola struktur organisasi kelas berbasis tree.
class Struktur:
    # Inisialisasi tree dengan root Ketua Kelas
    def __init__(self, nama_ketua):
        if nama_ketua:
            self.root = Anggota(nama_ketua, "Ketua Kelas")
        else:
            self.root = None

    # 1.
    # Menampilkan seluruh struktur organisasi secara rekursif
    def tampilkan_struktur(self, node=None, tingkat=0):
        if self.root is None:
            print("Struktur organisasi kosong.")
            return

        if node is None and tingkat == 0:
            node = self.root

        indentasi = "  " * tingkat + ("|_ " if tingkat > 0 else "")

        print(f"{indentasi}{node.nama} ({node.jabatan})")

        for bawahan in node.bawahan:
            self.tampilkan_struktur(bawahan, tingkat + 1)

    # Mencari node anggota berdasarkan nama secara rekursif
    def cari_bawahan(self, node, nama_target):
        if node is None:

```

```

        return None
    if node.nama.lower() == nama_target.lower():
        return node

    for bawahan in node.bawahan:
        hasil = self.cari_bawahan(bawahan, nama_target)
        if hasil:
            return hasil
    return None

# Mencari node parent (atasan langsung) dari anggota
def cari_parent(self, node, nama_target):
    if node is None:
        return None
    for bawahan in node.bawahan:
        if bawahan.nama.lower() == nama_target.lower():
            return node
        hasil = self.cari_parent(bawahan, nama_target)
        if hasil:
            return hasil
    return None

# 2.
# Menambahkan anggota baru sebagai bawahan dari atasan
def tambah_anggota(self, nama_atasan, nama_baru,
jabatan_baru):
    atasan = self.cari_bawahan(self.root, nama_atasan)
    if atasan:
        anggota_baru = Anggota(nama_baru, jabatan_baru)
        atasan.bawahan.append(anggota_baru)
        self.urutkan_bawahan(atasan.bawahan)
        print(f"Berhasil menambahkan {nama_baru} sebagai
{jabatan_baru}.")
    else:
        print(f"Gagal: Atasan dengan nama '{nama_atasan}'
tidak ditemukan!")

# Sorting bawahan secara alfabetis menggunakan insertion sort

```

```

def urutkan_bawahan(self, kumpulan_bawahan):
    n = len(kumpulan_bawahan)

    for i in range(1, n):
        temp = kumpulan_bawahan[i]
        j = i - 1

        while j >= 0 and kumpulan_bawahan[j].nama.lower() >
temp.nama.lower():
            kumpulan_bawahan[j + 1] = kumpulan_bawahan[j]
            j -= 1

        kumpulan_bawahan[j + 1] = temp

# 3.
# Menghapus anggota beserta seluruh bawahannya dari struktur
def hapus_anggota(self, nama_target, riwayat):
    if self.root is None:
        print("Struktur kosong.")
        return

    if self.root.nama.lower() == nama_target.lower():
        print("Peringatan: Tidak dapat menghapus Ketua Kelas
dari sini!")
        return

    parent = self.cari_parent(self.root, nama_target)
    if parent:
        target_node = self.cari_bawahan(self.root,
nama_target)
        riwayat.tambah(target_node)
        parent.bawahan = [b for b in parent.bawahan if
b.nama.lower() != nama_target.lower()]
        print(f"Berhasil menghapus '{nama_target}' beserta
seluruh bawahannya dari struktur.")
    else:
        print(f"Anggota dengan nama '{nama_target}' tidak
ditemukan!")

```

```

# 4.
# Mengubah data anggota (nama dan/atau jabatan)
def update_anggota(self, nama_target, nama_baru,
jabatan_baru):
    anggota = self.cari_bawahan(self.root, nama_target)
    if anggota:
        nama_lama = anggota.nama
        # Jika input tidak kosong, perbarui datanya
        if nama_baru.strip():
            anggota.nama = nama_baru
        if jabatan_baru.strip():
            anggota.jabatan = jabatan_baru
        print(f"Berhasil memperbarui data '{nama_lama}'.")
    else:
        print(f"Anggota dengan nama '{nama_target}' tidak
ditemukan!")

# 5.
# Mencari dan menampilkan detail informasi anggota
def cari_anggota(self, nama_target):
    hasil = self.cari_bawahan(self.root, nama_target)
    if hasil:
        print(f"\n===Data Anggota===")
        print(f>Nama      : {hasil.nama}")
        print(f>Jabatan  : {hasil.jabatan}")
        if hasil.bawahan:
            print("Bawahan:")
            for b in hasil.bawahan:
                print(f" - {b.nama} ({b.jabatan})")
        else:
            print("Status   : Tidak memiliki bawahan.")
    else:
        print("Anggota tidak ditemukan.")

# Fungsi utama program, menampilkan menu dan menangani input user
def main():

```

```

    # Pilihan awal untuk membuat struktur baru atau memuat
struktur sebelumnya

    print("\n=====\\n",
          "\\n===== Struktur Organisasi Kelas =====\\n",
          "\\n=====\\n",
          "1. Buat Struktur Organisasi Baru\\n",
          "2. Load Struktur Organisasi Sebelumnya\\n",)

    pilihan = input("Masukkan pilihan: ")

    # Inisialisasi struktur baru
    if pilihan == "1":
        nama_ketua = input("Masukkan nama ketua kelas: ")
        organisasi = Struktur(nama_ketua)
        riwayat = Riwayat()

    # Load struktur dari file JSON
    elif pilihan == "2":
        organisasi = Struktur(None)
        riwayat = Riwayat()
        valid = buka_file(organisasi)
        if not valid:
            print("Silakan buat struktur organisasi baru sebagai
gantinya.")
            nama_ketua = input("Masukkan nama ketua kelas: ")
            organisasi = Struktur(nama_ketua)
        else:
            print("Pilihan tidak valid!")
            return

    print("\\n--- Struktur Organisasi Kelas ---")
    organisasi.tampilkan_struktur()
    while True:
        print("\\n=====\\n",
              "\\n===== Struktur Organisasi Kelas =====\\n",
              "\\n=====\\n",
              "1. Tampilkan Struktur Organisasi\\n",
              "2. Tambah Anggota\\n",

```

```

        "3. Hapus Anggota\n",
        "4. Update Anggota\n",
        "5. Cari Anggota\n",
        "6. Simpan Struktur\n",
        "7. Load Struktur\n",
        "8. Tampilkan Riwayat Hapus\n",
        "0. Keluar"
    )

    pilihan = input("Masukkan pilihan: ")

    # IF ELSE INPUT USER

    if pilihan == "1":
        print("\n--- Struktur Organisasi Kelas ---")
        organisasi.tampilkan_struktur()
    elif pilihan == "2":
        nama_atasan = input("Masukkan nama atasan: ")
        nama_baru = input("Masukkan nama anggota baru: ")
        jabatan_baru = input("Masukkan jabatan anggota baru: ")

        organisasi.tambah_anggota(nama_atasan, nama_baru,
jabatan_baru)
    elif pilihan == "3":
        nama_target = input("Masukkan nama anggota yang ingin
dihapus: ")
        organisasi.hapus_anggota(nama_target, riwayat)
    elif pilihan == "4":
        nama_target = input("Masukkan nama anggota yang ingin
diubah: ")
        print("\n(Kosongkan dan tekan Enter pada bagian yang
tidak ingin diubah)")
        nama_baru = input("Masukkan nama baru: ")
        jabatan_baru = input("Masukkan jabatan baru: ")
        organisasi.update_anggota(nama_target, nama_baru,
jabatan_baru)
    elif pilihan == "5":

```

```
        nama_target = input("Masukkan nama anggota yang ingin  
dicari: ")  
        organisasi.cari_anggota(nama_target)  
    elif pilihan == "6":  
        simpan_file(organisasi)  
    elif pilihan == "7":  
        buka_file(organisasi)  
    elif pilihan == "8":  
        riwayat.tampilkan()  
    elif pilihan == "0":  
        break  
    else:  
        print("Pilihan tidak valid!")  
  
if __name__ == "__main__":  
    main()
```